

Main.py

```
import streamlit as st
```

```
st.set_page_config(page_title="Event Recommender", layout="centered")
```

```
st.markdown("""
```

```
<style>
```

```
.btn-container {
```

```
    display: flex;
```

```
    justify-content: space-around;
```

```
    margin-top: 60px;
```

```
}
```

```
.btn {
```

```
    display: inline-block;
```

```
    padding: 20px 40px;
```

```
    background: linear-gradient(135deg, #4c6ef5, #15aabf);
```

```
    color: white;
```

```
    border-radius: 20px;
```

```
    font-size: 20px;
```

```
    font-weight: bold;
```

```
    text-decoration: none;
```

```
    transition: all 0.3s ease-in-out;
```

```
    box-shadow: 0 8px 16px rgba(0,0,0,0.2);
```

```
}
```

```
.btn:hover {
```

```
    background: linear-gradient(135deg, #15aabf, #4c6ef5);
```

```
    transform: translateY(-5px);
```

```
}
```

```
</style>
```

```
""", unsafe_allow_html=True)
```

```
st.title(" 🎉 Welcome to the Event Recommendation System")
```

```
st.markdown("""  
    <div class="btn-container">  
        <a href="/Concerts" class="btn"> 🎵 Concerts</a>  
        <a href="/Sports" class="btn"> 🏟️ Sports</a>  
        <a href="/Festivals" class="btn"> 🎪 Festivals</a>  
    </div>  
""", unsafe_allow_html=True)
```

Concerts.py

```
import streamlit as st  
import pandas as pd  
import streamlit.components.v1 as components  
  
st.set_page_config(page_title="Concert Recommendations", layout="wide")  
  
st.title(" 🎵 Concert Recommendations")  
  
# Load the CSV  
try:  
    df = pd.read_csv("concerts.csv")  
except FileNotFoundError:  
    st.error(" ❌ 'concerts.csv' not found.")  
    st.stop()  
  
# Dropdown filters  
cities = sorted(df["city"].dropna().unique().tolist())  
levels = sorted(df["level"].dropna().unique().tolist())
```

```

subtypes = sorted(df["subtype"].dropna().unique().tolist())

city = st.selectbox("Select City", options=[""] + cities)
level = st.selectbox("Select Level", options=[""] + levels)
subtype = st.selectbox("Select Subtype", options=[""] + subtypes)

# Filter logic
filtered = df.copy()
if city:
    filtered = filtered[filtered["city"] == city]
if level:
    filtered = filtered[filtered["level"] == level]
if subtype:
    filtered = filtered[filtered["subtype"] == subtype]

# Sort by popularity
filtered = filtered.sort_values(by="popularity", ascending=False)

# Determine whether to show top 10 or filtered results
if not city and not level and not subtype:
    display_df = df.sort_values(by="popularity", ascending=False).head(10)
    st.subheader("🔥 Top 10 Trending Concerts")
else:
    display_df = filtered
    st.subheader("🎯 Recommended Concerts")

if display_df.empty:
    st.warning("No concerts found matching the selected filters.")
else:
    # Grid layout with 3 per row

```

```
cards_html = ""
<style>
.grid-container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  justify-content: flex-start;
  padding-top: 10px;
}
.event-card {
  background-color: #ffffff;
  border-radius: 15px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  padding: 20px;
  width: calc(33% - 20px);
  box-sizing: border-box;
  transition: transform 0.2s ease, box-shadow 0.3s ease;
}
.event-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 8px 20px rgba(0,0,0,0.15);
}
.event-title {
  font-size: 18px;
  font-weight: bold;
  color: #3f51b5;
}
.event-info {
  font-size: 14px;
  color: #333;
```

```

        margin-top: 6px;
    }
</style>
<div class="grid-container">
    ""

```

```

for _, row in display_df.iterrows():
    cards_html += f"""
    <div class="event-card">
        <div class="event-title">{row['name']}</div>
        <div class="event-info">📅 {row['date']}</div>
        <div class="event-info">📍 {row['city']}</div>
        <div class="event-info">🏷️ {row['subtype']}</div>
        <div class="event-info">🌐 {row['level']}</div>
        <div class="event-info">🔥 Popularity: {row['popularity']}</div>
    </div>
    """

cards_html += "</div>"
components.html(cards_html, height=800, scrolling=True)

```

Contact.py

```
import streamlit as st
```

```
st.set_page_config(page_title="Contact Us", layout="centered")
```

```
st.title("📞 Contact Information")
```

```
st.markdown("Feel free to reach out to us using the details below:")
```

```
# Divider
```

```
st.markdown("---")
```

```
# Contact Info Section
```

```
st.markdown("""
```

```
### 📞 Get in Touch
```

```
- **Email**: ultronmegatron19@gmail.com
```

```
- **Phone**: +91 8374466035
```

```
- **Address**:
```

```
    Event Recommender Inc.
```

```
    CMR Engineering College Medchal-500401.
```

```
""")
```

```
# Optional: Footer or support notice
```

```
st.markdown("---")
```

```
st.info("We typically respond to emails within 24 hours.")
```

Sports.py

```
import streamlit as st
```

```
import pandas as pd
```

```
import streamlit.components.v1 as components
```

```
st.set_page_config(page_title="Sports Recommendations", layout="wide")
```

```
st.title("🏆 Sports Event Recommendations")
```

```
# Load CSV
```

```
try:
```

```
    df = pd.read_csv("sports.csv")
```

```
except FileNotFoundError:
```

```
    st.error("❌ 'sports_events.csv' not found.")
```

```

st.stop()

# Filters
cities = sorted(df["city"].dropna().unique().tolist())
levels = sorted(df["level"].dropna().unique().tolist())
main_types = ["Indoor", "Outdoor"]

city = st.selectbox("Select City", options=[""] + cities)
level = st.selectbox("Select Level", options=[""] + levels)
main_type = st.radio("Select Event Type", options=main_types, horizontal=True)

# Filter subtypes based on Indoor/Outdoor
if main_type == "Indoor":
    subtypes = sorted(df[df["subtype"].str.lower().str.contains("indoor")]["subtype"].unique())
else:
    subtypes = sorted(df[df["subtype"].str.lower().str.contains("outdoor")]["subtype"].unique())

# Remove the "Indoor" or "Outdoor" part from subtype for display
subtypes = [subtype.replace("Indoor ", "").replace("Outdoor ", "") for subtype in subtypes]

subtype = st.selectbox("Select Sport", options=[""] + subtypes)

# Apply Filters
filtered = df.copy()
if city:
    filtered = filtered[filtered["city"] == city]
if level:
    filtered = filtered[filtered["level"] == level]
if subtype:
    filtered = filtered[filtered["subtype"].str.contains(subtype, case=False)]

```

```
filtered = filtered.sort_values(by="popularity", ascending=False)
```

```
# Display Results
```

```
if not city and not level and not subtype:
```

```
    display_df = df.sort_values(by="popularity", ascending=False).head(10)
```

```
    st.subheader("🔥 Top 10 Trending Sports Events")
```

```
else:
```

```
    display_df = filtered
```

```
    st.subheader("🎯 Recommended Sports Events")
```

```
if display_df.empty:
```

```
    st.warning("No events found matching the selected filters.")
```

```
else:
```

```
    # Grid Layout
```

```
    cards_html = ""
```

```
<style>
```

```
.grid-container {
```

```
    display: flex;
```

```
    flex-wrap: wrap;
```

```
    gap: 20px;
```

```
    justify-content: flex-start;
```

```
    padding-top: 10px;
```

```
}
```

```
.event-card {
```

```
    background-color: #ffffff;
```

```
    border-radius: 15px;
```

```
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
```

```
    padding: 20px;
```

```
    width: calc(33% - 20px);
```

```

    box-sizing: border-box;

    transition: transform 0.2s ease, box-shadow 0.3s ease;
}

.event-card:hover {

    transform: translateY(-5px);

    box-shadow: 0 8px 20px rgba(0,0,0,0.15);
}

.event-title {

    font-size: 18px;

    font-weight: bold;

    color: #2e7d32;
}

.event-info {

    font-size: 14px;

    color: #333;

    margin-top: 6px;
}

</style>

<div class="grid-container">

    ""

```

```

for _, row in display_df.iterrows():

    # Display only the sport name without "Indoor" or "Outdoor"

    event_subtype = row['subtype'].replace("Indoor ", "").replace("Outdoor ", "")

    cards_html += f"""

    <div class="event-card">

        <div class="event-title">{row['name']}</div>

        <div class="event-info">📅 {row['date']}</div>

        <div class="event-info">📍 {row['city']}</div>

        <div class="event-info">🏊 {event_subtype}</div>

```

```

<div class="event-info">🌐 {row['level']}</div>

<div class="event-info">🔥 Popularity: {row['popularity']}</div>

</div>

"""

cards_html += "</div>"

components.html(cards_html, height=800, scrolling=True)

```

Festivals.py

```

import streamlit as st
import pandas as pd
import streamlit.components.v1 as components

st.set_page_config(page_title="Festival Recommendations", layout="wide")
st.title("🏠 Festival Event Recommendations")

# Load CSV
try:
    df = pd.read_csv("festivals.csv")
except FileNotFoundError:
    st.error("❌ 'festivals.csv' not found.")
    st.stop()

# Filters
cities = sorted(df["city"].dropna().unique().tolist())
levels = sorted(df["level"].dropna().unique().tolist())
types = sorted(df["subtype"].dropna().unique().tolist())

city = st.selectbox("Select City", options=[""] + cities)
level = st.selectbox("Select Level", options=[""] + levels)

```

```
subtype = st.selectbox("Select Festival Type", options=[""] + types)
```

```
# Apply Filters
```

```
filtered = df.copy()
```

```
if city:
```

```
    filtered = filtered[filtered["city"] == city]
```

```
if level:
```

```
    filtered = filtered[filtered["level"] == level]
```

```
if subtype:
```

```
    filtered = filtered[filtered["subtype"] == subtype]
```

```
filtered = filtered.sort_values(by="popularity", ascending=False)
```

```
# Display Results
```

```
if not city and not level and not subtype:
```

```
    display_df = df.sort_values(by="popularity", ascending=False).head(10)
```

```
    st.subheader("🔥 Top 10 Trending Festivals")
```

```
else:
```

```
    display_df = filtered
```

```
    st.subheader("🎯 Recommended Festivals")
```

```
if display_df.empty:
```

```
    st.warning("No festivals found matching the selected filters.")
```

```
else:
```

```
    # Grid Layout for Events
```

```
    cards_html = ""
```

```
<style>
```

```
.grid-container {
```

```
    display: flex;
```

```
    flex-wrap: wrap;
```

```
    gap: 20px;
    justify-content: flex-start;
    padding-top: 10px;
  }
  .event-card {
    background-color: #ffffff;
    border-radius: 15px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
    padding: 20px;
    width: calc(33% - 20px);
    box-sizing: border-box;
    transition: transform 0.2s ease, box-shadow 0.3s ease;
  }
  .event-card:hover {
    transform: translateY(-5px);
    box-shadow: 0 8px 20px rgba(0,0,0,0.15);
  }
  .event-title {
    font-size: 18px;
    font-weight: bold;
    color: #8e44ad;
  }
  .event-info {
    font-size: 14px;
    color: #333;
    margin-top: 6px;
  }
</style>
<div class="grid-container">
  ""
```

```

for _, row in display_df.iterrows():
    cards_html += f"""
    <div class="event-card">

        <div class="event-title">{row['name']}</div>

        <div class="event-info"> 📅 {row['date']}</div>

        <div class="event-info"> 📍 {row['city']}</div>

        <div class="event-info"> 🏷️ {row['subtype']}</div>

        <div class="event-info"> 🌐 {row['level']}</div>

        <div class="event-info"> 🔥 Popularity: {row['popularity']}</div>

    </div>

    """

cards_html += "</div>"
components.html(cards_html, height=800, scrolling=True)

```

Styles.css

```

.title {
    text-align: center;
    font-size: 48px;
    font-weight: bold;
    margin-bottom: 20px;
    color: #2E86C1;
}

.navbar {
    display: flex;
    justify-content: center;
    gap: 25px;
    margin-bottom: 40px;
}

```

```
    font-size: 20px;  
}
```

```
.navbar a {  
    text-decoration: none;  
    padding: 10px 20px;  
    background-color: #3498db;  
    color: white;  
    border-radius: 10px;  
    transition: background-color 0.3s ease;  
}
```

```
.navbar a:hover {  
    background-color: #1abc9c;  
}
```